Introduction to PID Control

Adapted from the presentation by Brad Schofield

DRAFT

What is PID Control?

- Let's take a step back… What is **control**?
- **Control** is just making a dynamic process behave in the way we want
- We need 3 things to do this:
  - A way to **influence** the process
  - A way to see how the process **behaves**
  - A way to **define** how we want it to behave

DRAFT

Defining behaviour

- We usually specify a **value** we want some output of the system to have
  - Usually called the **Setpoint (SP)**
- Can be the temperature of a room, the level in a tank, the flow rate in a pipe…
- The value can be fixed, or may change with time…

DRAFT

Influencing the process

- We need some kind of **control input** which can create changes in the behavior of the process
- Can be a heater, a valve, a pump…
- Typically it is **not** the same physical quantity as what we are controlling

DRAFT

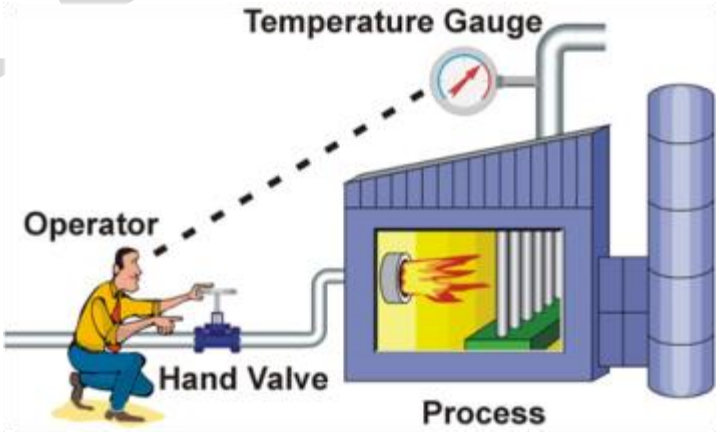
Observing behavior

- If we knew exactly how the process worked, we would know what the output would be for a given control input…
- Most of the time we don't know exactly, so we need to **measure** what the process does
  - Usually called **Measured Variable (MV)** or **Process Variable (PV)**

DRAFT

Feedback Control

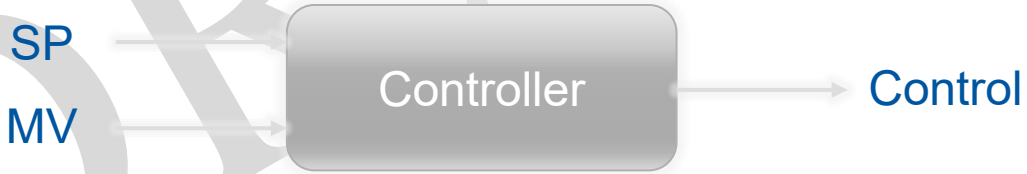
- Now we have a measurement (**MV**), some value that we want it to be (**SP**), and some way to make changes to the process (**control input**)
- We can '**close the loop**'



DRAFT

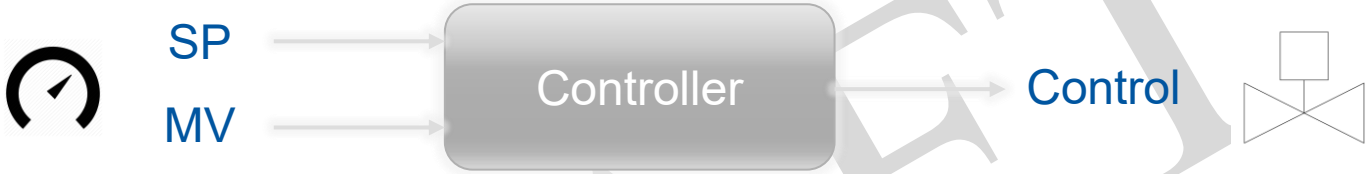
The Controller as a System

- Now we can see that any controller can be thought of as a system that takes a **setpoint** and a **measured value** as inputs, and gives a **control** signal as an output



DRAFT

The Controller as a System



- The controller needs to convert two signals of one physical quantity (such as temperature) into one signal of another (such as valve position)

DRAFT

The Controller as a System

- We know that the process is a **dynamic** system:
  - Its outputs depend on **current** inputs as well as its **past** state
- For the controller to deal with this, it makes sense that it should be a **dynamic** system too

DRAFT

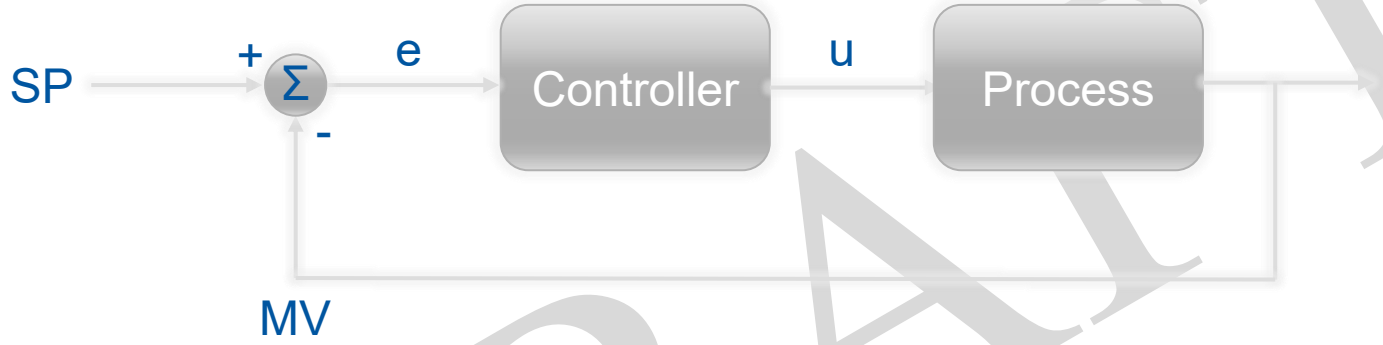
The Error Signal

- Very often we can think of the controller acting on the **difference** between SP and MV:



DRAFT

The Closed Loop



This is the ‘classic’ closed loop block diagram representation of a control system

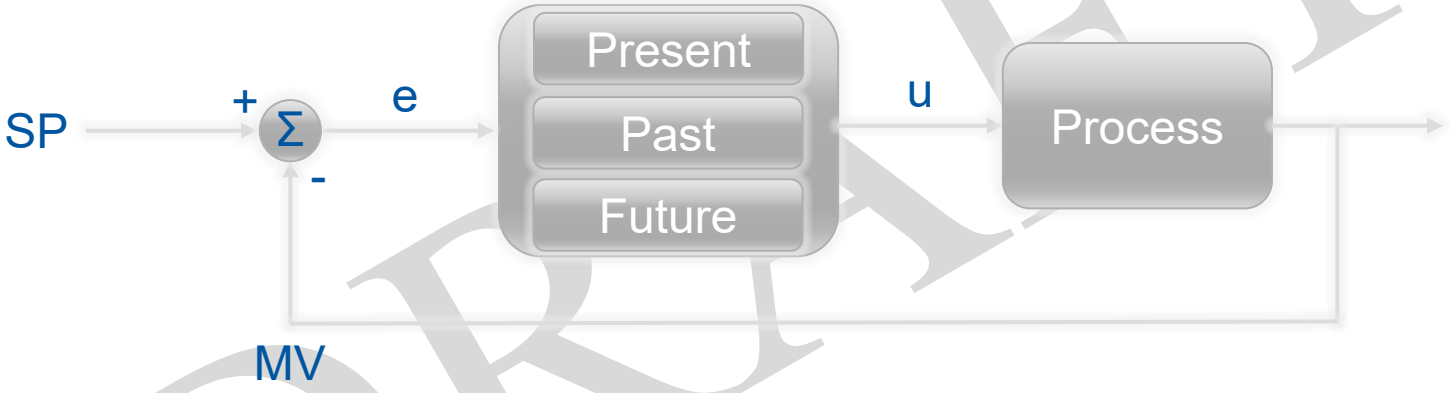
DRAFT

A Dynamic Controller

- We said that since the process is dynamic (dependent on inputs made at different times), it makes sense that the controller should be too
- How do we usually think of time?
  - ‘Present’
  - ‘Past’
  - ‘Future’

DRAFT

Splitting the Controller



DRAFT

The 'Present'

- This part of the controller is only concerned with what the **error** is **now**
- Let's take a simple law: let the control signal be **proportional** to the error:

$$\boldsymbol{u = k_p \times e}$$

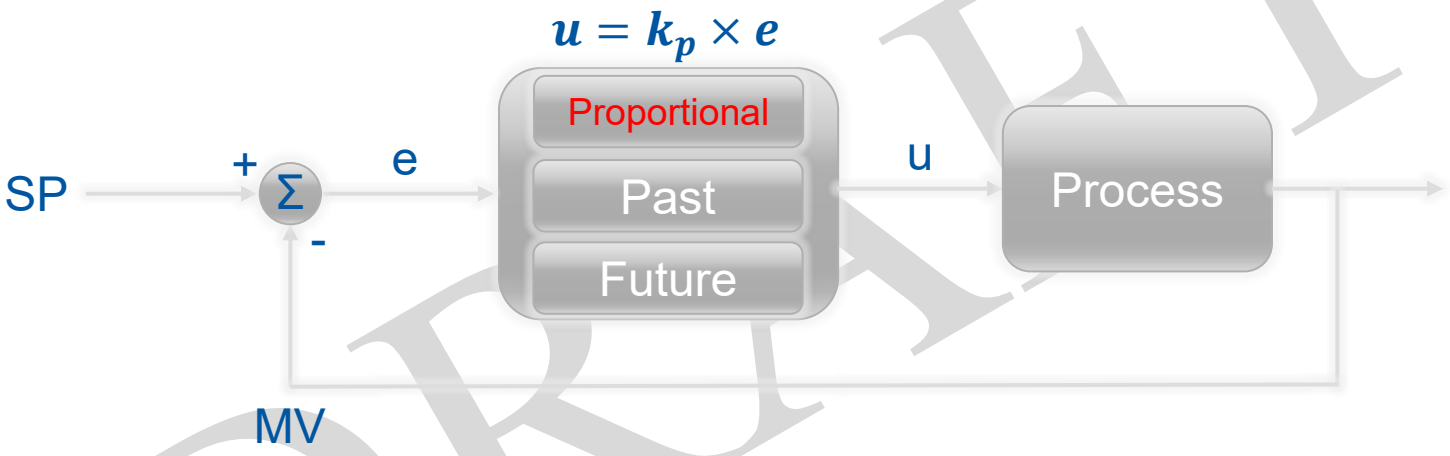
DRAFT

Proportional Control

- This is what is referred to as **proportional control.** The control action at any instant is the same as a constant times the error at the same instant
- The constant k_p is the **Proportional Gain,** and is the first of our controller's parameters

DRAFT

Proportional Control



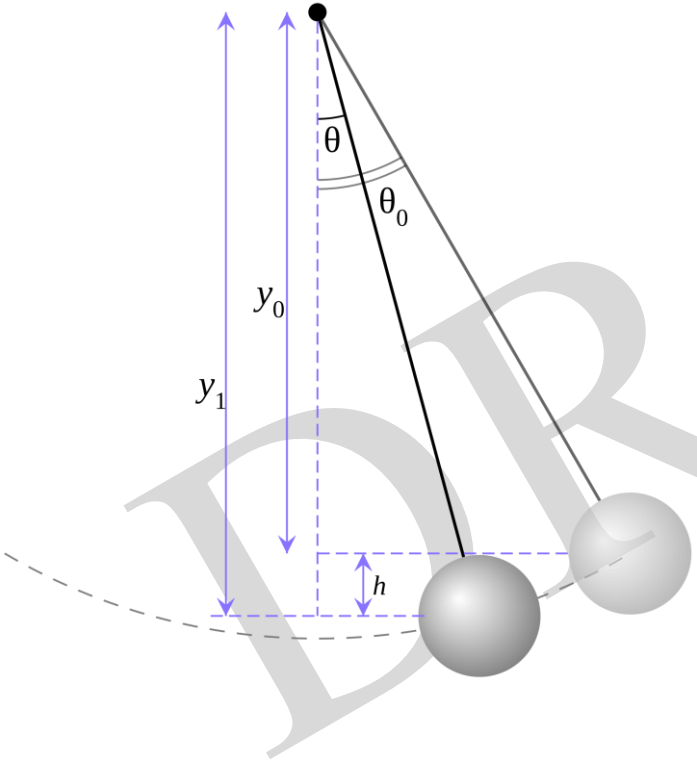
DRAFT

Is Proportional Control enough?

- Intuitively it seems like it should be fine on its own: when the error is big, the control input is big to correct it. As the error reduced so does the control input.
- But there are problems…

DRAFT

Problem 1



Think of a pendulum.

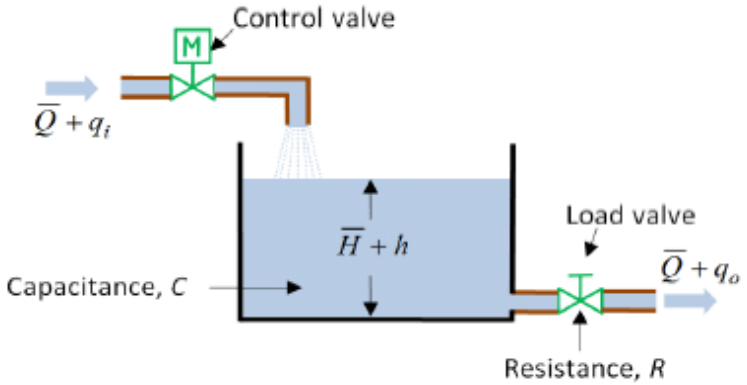
If the setpoint is hanging straight down, then gravity acts as a proportional controller for the position…

Pendulum will **oscillate**!

DRAFT

Problem 2

- What happens when the error is zero?
- Control input is zero.
- Causes problems if we need to have a nonzero control value while at our setpoint



DRAFT

Problem 2: steady state error

This problem is normally called **steady state error**

It's a confusing name. The issue is just that the controller can't produce any output when the error is zero.

Easiest to see for tank level control: If there is a constant flow out of the tank, the controller must provide the same flow in, while the level is at the setpoint.

DRAFT

Problem 2: steady state error

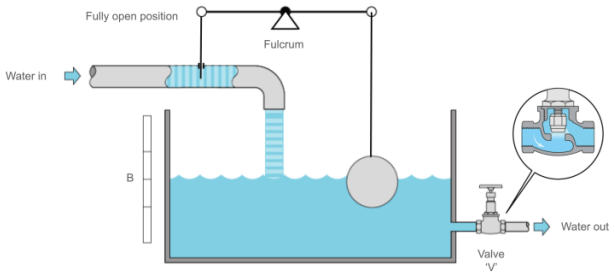


Fig. 5.2.6 Valve open

With P control, once the error has reached a value where $k_p \times e$ is equal to the flow out, the level will stabilize. But it will be different to the setpoint

DRAFT

P control problem summary

- Problem 1: oscillations
  - P control will give us oscillations in some processes, regardless of the value of the gain parameter.
- Problem 2: steady state error
  - P control cannot give us a nonzero value of the control at zero error for some types of process

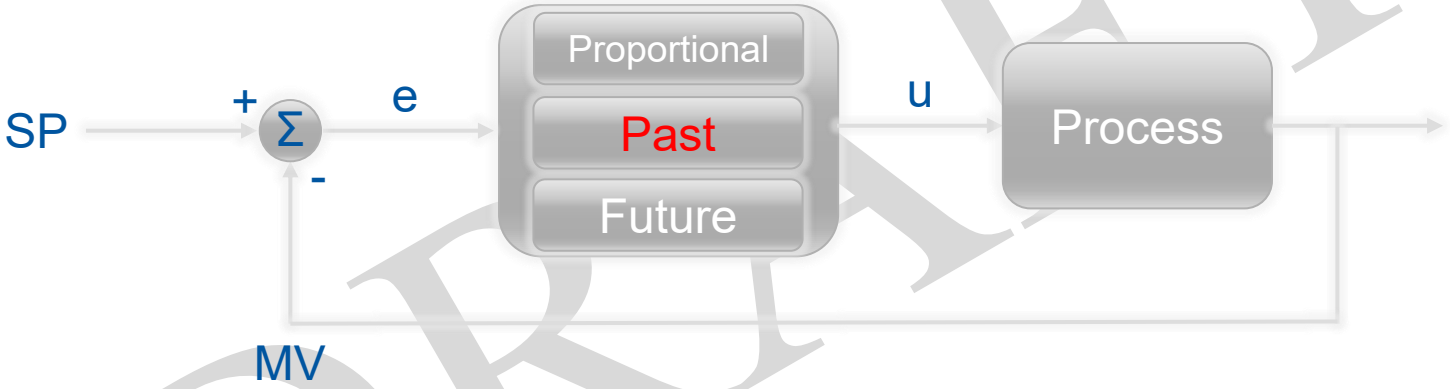
DRAFT

Solving P control's problems

- How to get rid of steady state error?
- Let's ignore the present for the moment and concentrate on what has happened in the **past**

DRAFT

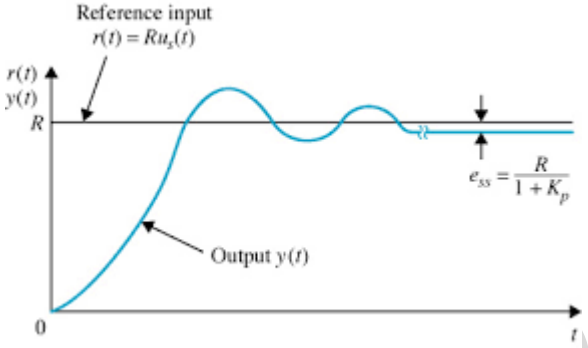
Solving steady state error: 'the Past'

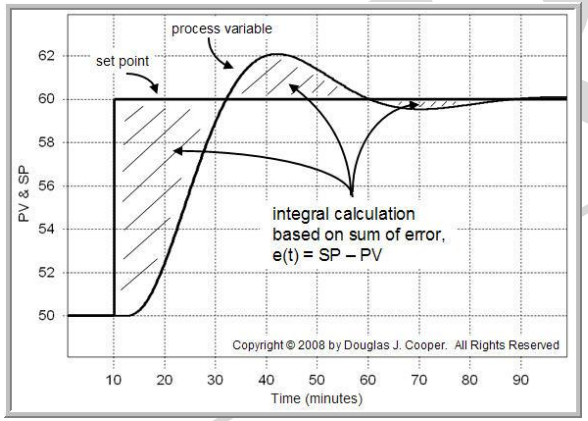


- Let's look at the error in the **past**

DRAFT

Solving steady state error





We can examine how the controller error has evolved in the **past**

If we sum up the past values of the error, we can get a value that increases when there is a constant error

DRAFT

Integral Action

We can let the control be given by the sum of past values of the error, scaled by some gain.

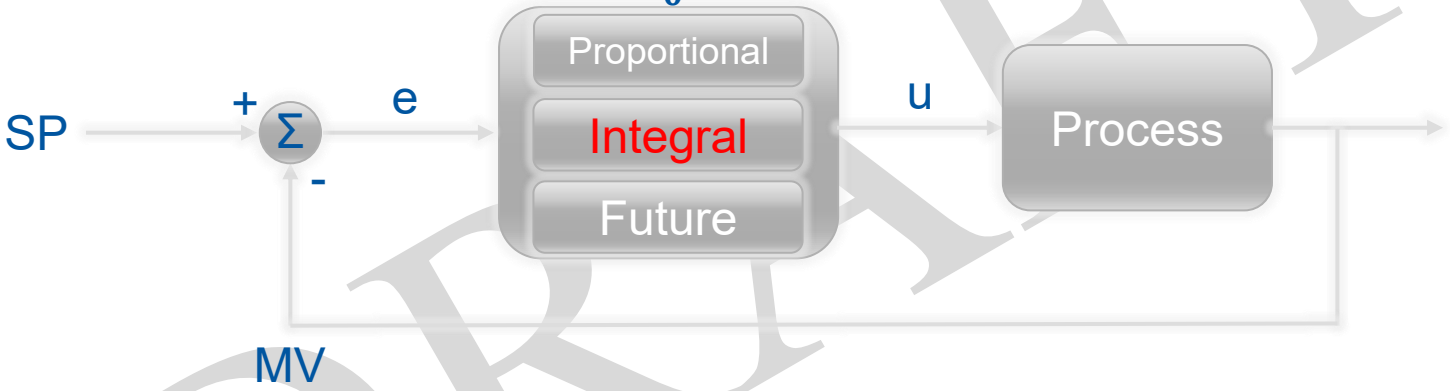
In continuous time the sum is an **integral**:

$$\boldsymbol{u = k_i \int_0^t e(\tau) d\tau}$$

DRAFT

Integral Action

$$u = k_i \int_0^t e(\tau) d\tau$$



DRAFT

Integral gain, Integral time

Here is where confusion can start…

We have an **integral gain k_i** which converts an integrated error to a control signal

We would actually like to parameterize this as a **time**, as in how fast we can remove a steady state error

DRAFT

Integral gain, Integral time

Let's rewrite:

$$u = k_i \int_0^t e(\tau) d\tau$$

As:

$$u = \frac{k_p}{T_i} \int_0^t e(\tau) d\tau$$

is our Proportional Gain, and T_i is our **Integral Time**

DRAFT

The Integral time

Why do we use both k_p and T_i here?

$$u = \frac{k_p}{T_i}\int_0^t e(\tau)d\tau$$

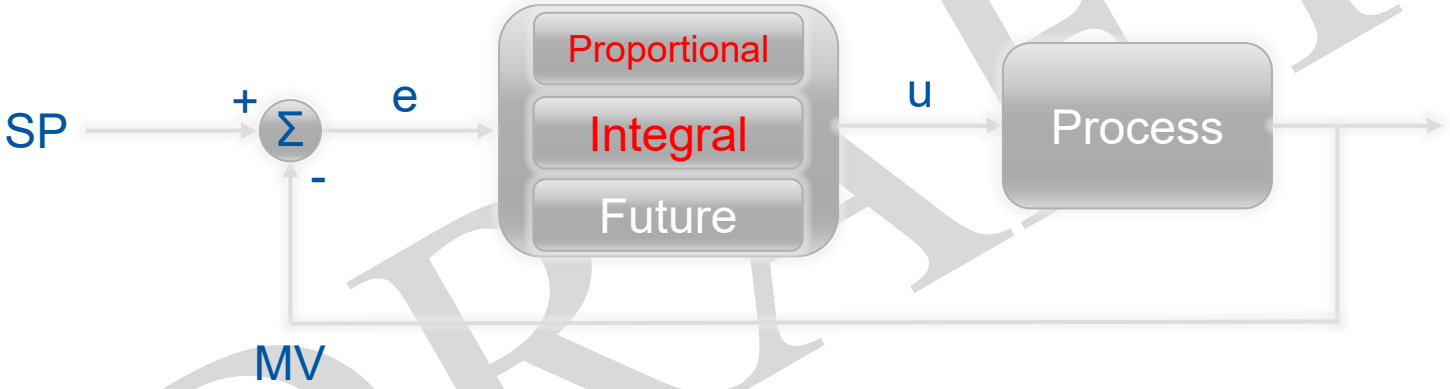
Let's add the proportional and integral parts:

$$u = k_p e(t) + \frac{k_p}{T_i}\int_0^t e(\tau)d\tau$$

DRAFT

Proportional and Integral Controller

$$u = k_p e(t) + \frac{k_p}{T_i}\int_0^t e(\tau)d\tau$$



DRAFT

Proportional and Integral Controller = PI Controller

We can rewrite the control law:

$$\boldsymbol{u = k_p\left(e(t) + \frac{1}{T_i}\int_0^t e(\tau)d\tau\right)}$$

$\boldsymbol{k_p}$ is the gain parameter, $\boldsymbol{T_i}$ is the time it takes to fix a steady state error

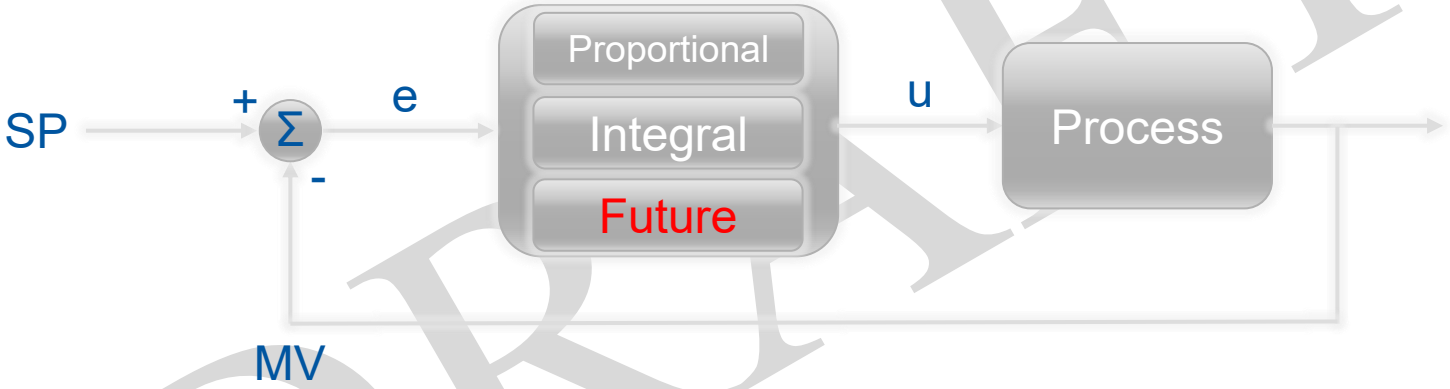
DRAFT

Solving P control's problems revisited

- We solved the steady state error by adding **integral action** (summing the past)
- How can we solve the oscillation problem?
- Let's look at the future!

DRAFT

Solving oscillations: ‘the Future’



- Let’s look at the error in the **future**

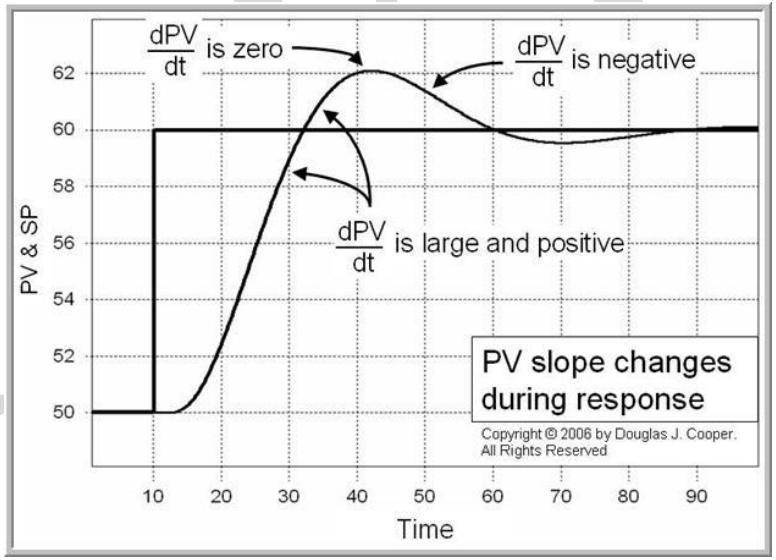
DRAFT

Solving oscillations: ‘the Future’

How do we predict the future of the error?

Look at its **gradient**!

If the gradient (the time derivative) of the error is in a direction that makes the error smaller, we can reduce the control input

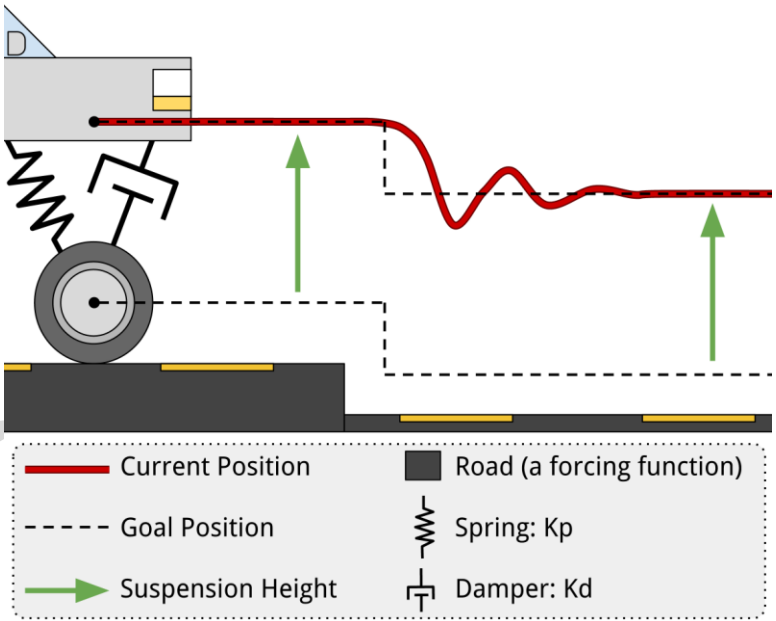


DRAFT

Damping

It can be easier to think of this as **damping,** something that resists velocity

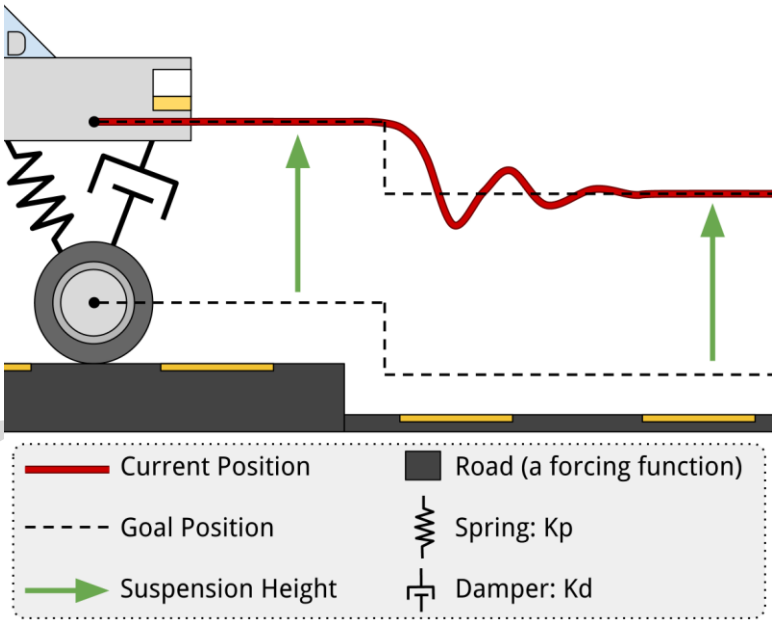
Think of the wheel on your car…



DRAFT

Damping

The spring is a proportional controller for the wheel position. The damper adds a **derivative action** by opposing the **velocity** of the wheel



DRAFT

Derivative action

Let's let the control be dependent on the derivative of the error:

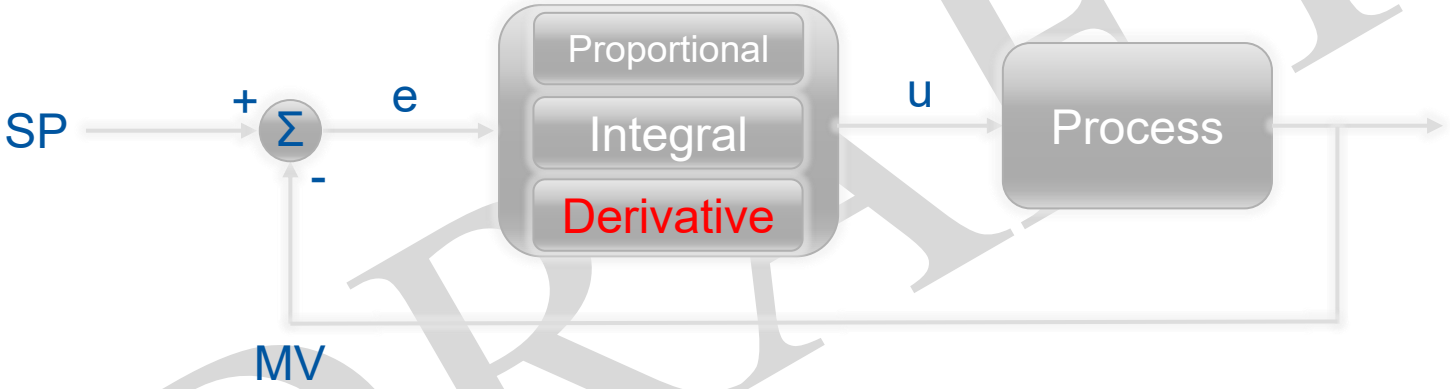
$$\boldsymbol{u = k_d \frac{de(t)}{dt}}$$

Here k_d is the derivative gain. Let's again split this into $k_p T_d$, where T_d is the **derivative time**

DRAFT

Derivative action

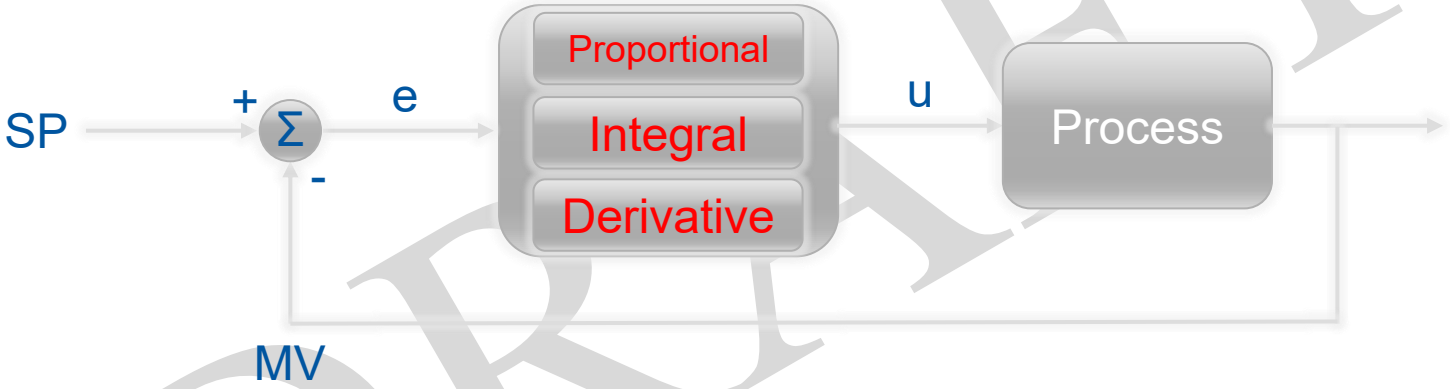
$$u = k_p T_d \frac{de(t)}{dt}$$



DRAFT

PID Controller!

$$u = k_p e(t) + \frac{k_p}{T_i}\int_0^t e(\tau)d\tau + k_p T_d \frac{de(t)}{dt}$$



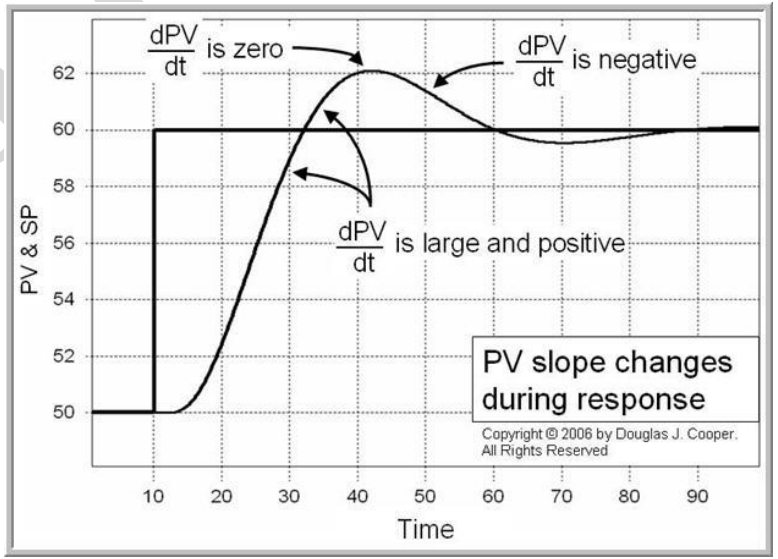
DRAFT

Derivative Time

Why do we want T_d as a parameter?

We can think of it as how far ahead we want to predict!

Easier to relate to process



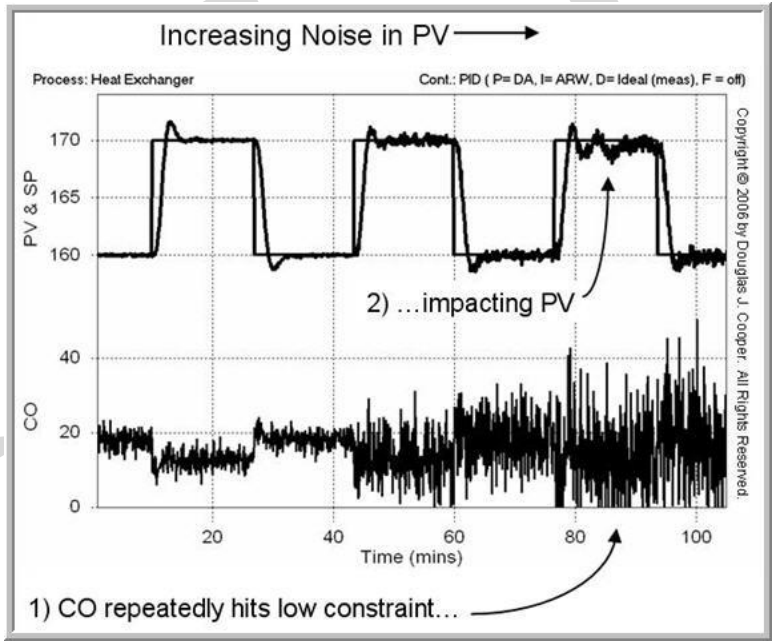
DRAFT

Problems with Derivative action

We know that a derivative amplifies quick changes.

We can get problems if MV is noisy.

Solution is to add low pass filter



DRAFT

Derivative with filter

We already have:

$$u = k_p T_d \frac{de(t)}{dt}$$

Equations will get messy if we add a filter in time domain! Let's use Laplace! Then we can use algebra instead of calculus.

DRAFT

Derivative with filter

Laplace transform frequency variable s is also an operator. Multiplication by s is derivation, and division is integration. So the derivative part is now

$$U(s) = k_p T_d s E(s)$$

DRAFT

Derivative with filter

We add a low pass filter:

$$U(s) = \frac{k_p T_d s}{1 + \frac{T_d}{T_{ds}} s} E(s)$$

Now we have another parameter T_{ds} which is the filter bandwidth. So introducing derivative action requires two more parameters!

DRAFT

Full PID equation

The complete PID controller now looks like:

$$U(s) = k_p E(s) + \frac{k_p E(s)}{T_i s} + \frac{k_p T_d s}{1 + \frac{T_d}{T_{ds}} s} E(s)$$

DRAFT

Full PID equation

This simplifies to:

$$U(s) = k_p\left(1 + \frac{1}{T_i s} + \frac{T_d s}{1 + \frac{T_d}{T_{ds}} s}\right) E(s)$$

DRAFT

ISA PID Form

This is the ISA 'standard form' for a PID

$$\frac{U(s)}{E(s)} = k_p \left(1 + \frac{1}{T_i s} + \frac{T_d s}{1 + \frac{T_d}{T_{ds}} s} \right)$$

We have one gain, and three time constants

DRAFT

Simulation using MATLAB

- Setpoint: unit step signal

$$w = 1_{\geq 0} \Rightarrow W(s) = \frac{1}{s}$$

- Process

$$P(s) = \frac{1}{10s^2 + 7s + 1}$$

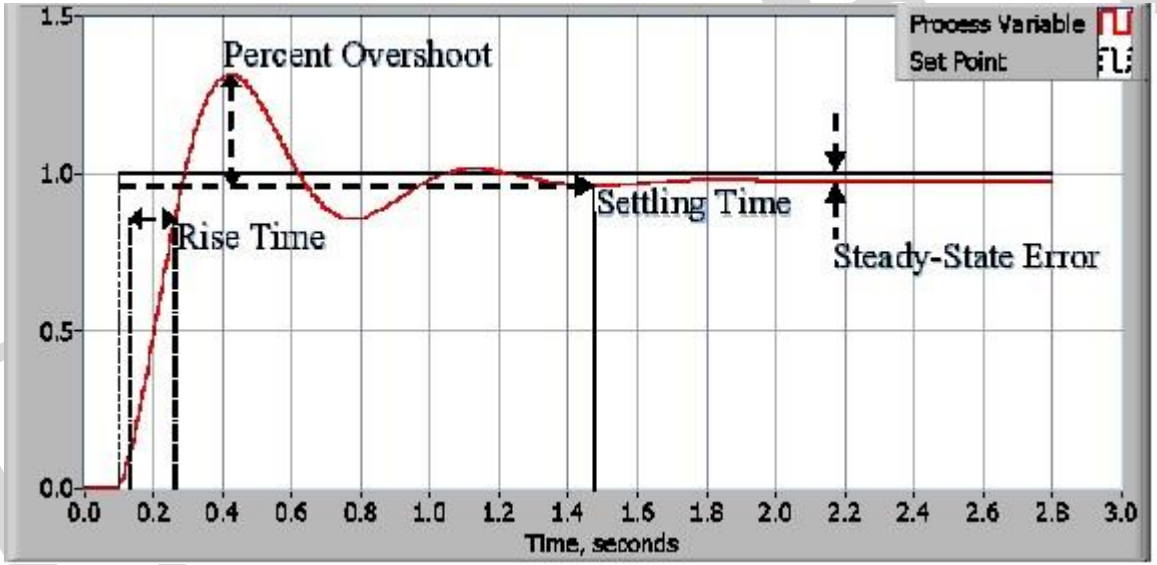
- Control: PID

$$C(s) = K_p \left(1 + \frac{1}{T_i s} + T_d s\right)$$

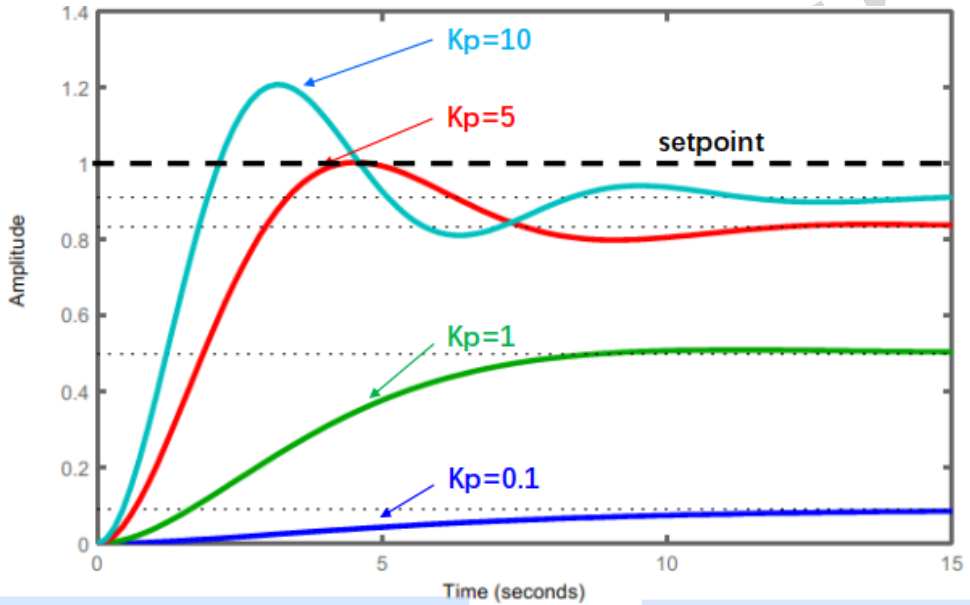
DRAFT

Simulation using MATLAB

- Defintion of Terminlogies



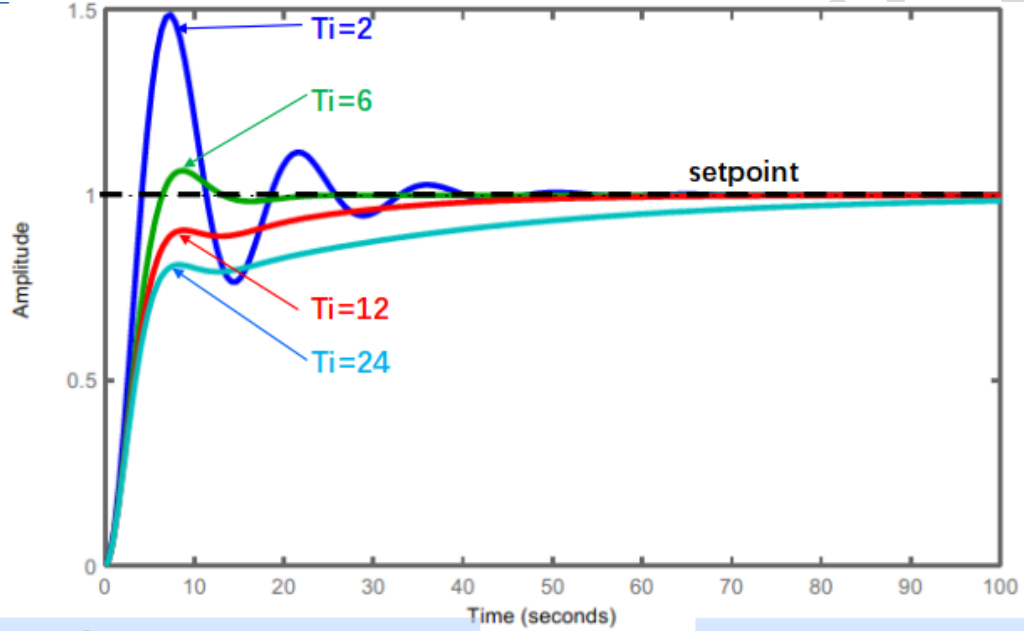
Simulation using MATLAB




```
%P control
Plt = tf(1,[10,7,1]);
Kp = [0.1,1,5,10];
for k = 1:4
    sys = feedback(Kp(k)*Plt,1);
    step(sys),hold on
end
```

- ✓ Kp increases, the response speed of the system increases, the overshoot of the closed-loop system increases, and the steady-state error decreases.
- ✓ Kp large enough, the closed-loop system becomes unstable

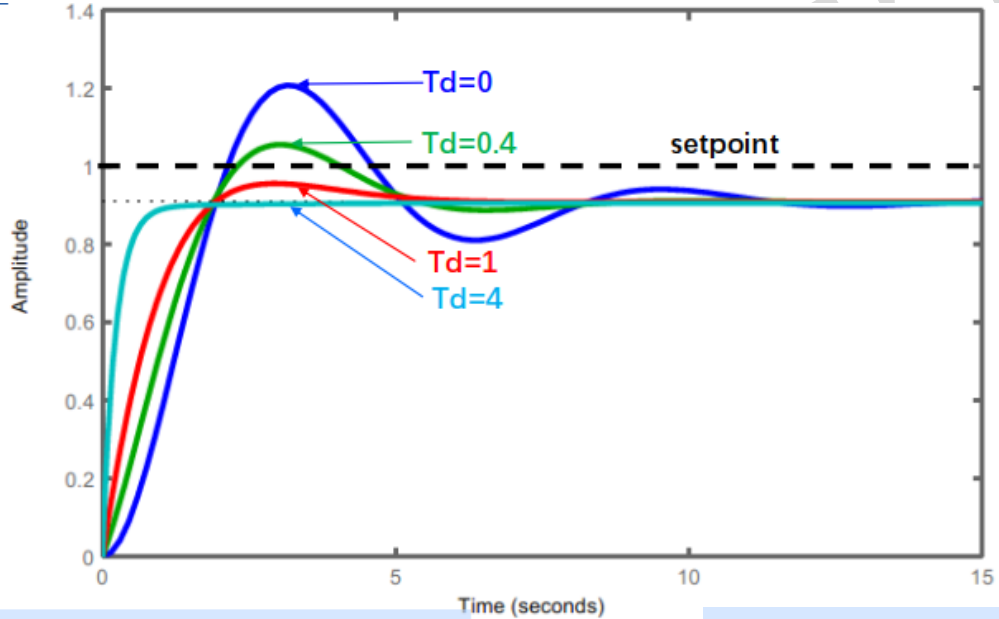
Simulation using MATLAB




```
%PI control:
Plt = tf(1,[10,7,1]);
Kp = 2; Ti = [2,6,12,24];
for m = 1:4
    Cpi = tf([Kp,Kp/Ti(m)],[1,0]);
    sys = feedback(Cpi*Plt,1);
    step(sys); hold on;
end
```

- ✓ No steady-state error in the step response
- ✓ Ti increases, the overshoot tends to be smaller, but the speed of response tends to be slower.

Simulation using MATLAB




```
%PD control:
Plt = tf(1,[10,7,1]);
Kp = 10; Td = [0,0.4,1,4];
for m = 1:4
    Cpd = tf([Kp*Td(m),Kp],[0,1]);
    sys = feedback(Cpd*Plt,1);
    step(sys); hold on;
end
```

- ✓ Td increases, the response has a smaller overshoot but similar settling time

- Some intuitions about effects of the terms:
  - ✓ **Increasing K_p**: Same amount of error generates a proportionally larger amount of control, makes system faster, but overshoot more (less stable).
  - ✓ **Increasing K_i**: Control effort builds as error is integrated over time, helps reduce steady state error, but can be slow to respond.
  - ✓ **Increasing K_d**: Allows controller to anticipate an increase in error, adds damping into the system (reduces overshoot), can amplify noise.

DRAFT

TABLE I
EFFECTS OF INDEPENDENT P, I, AND D TUNING

Closed-Loop Response	Rise Time	Overshoot	Settling Time	Steady-State Error	Stability
Increasing K_P	Decrease	Increase	Small Increase	Decrease	Degrade
Increasing K_I	Small Decrease	Increase	Increase	Large Decrease	Degrade
Increasing K_D	Small Decrease	Decrease	Decrease	Minor Change	Improve

- These guidelines do not hold for all situations.
- For systems that are not canonical first and second order, need to use trial and error.

Summary

- The controller performs the PID mathematical functions on the error and applies their sum to a process.
- We can build a PID controller that works well in practice in most situations without knowing control theory.

	Math Function	Effect on Control System
P Proportional	$K_{\mathrm{p}}e(t)$	Typically the main drive in a control loop, K_{p} reduces a large part of the overall error.
I Integral	$K_{\mathrm{i}}\int_0^t e(s)\,\mathrm{d}s$	Reduces the final error in a system. Summing even a small error over time produces a drive signal large enough to move the system toward a smaller error.
D Derivative	$K_{\mathrm{d}}\frac{\mathrm{d}e(t)}{\mathrm{d}t}$	Counteracts the K_{p} and K_{i} terms when the output changes quickly. This helps reduce overshoot and ringing. No effect on final error.

Resources

- Matlab video series: Understanding PID Control

https://www.mathworks.com/videos/series/understanding-pid-control.html

- Controlling Self Driving Cars

https://www.youtube.com/watch?v=4Y7zG48uHRo&t=210s

- Simple Examples of PID Control

https://www.youtube.com/watch?v=XfAt6hNV8XM

DRAFT

Manual PID Tuning

DRAFT

P, PI, PD, PID?

For the complete PID controller we have 4 parameters k_p, T_i, T_d, T_{ds}

But we can also choose to use only parts of the controller, for example just, PI, giving 2 parameters to choose.

How do we know when to use what?

DRAFT

The Process Model

We now know exactly what's inside our controller, but so far we haven't said anything about what's inside our process!

To be able to tune the controller, we need to know something about it.

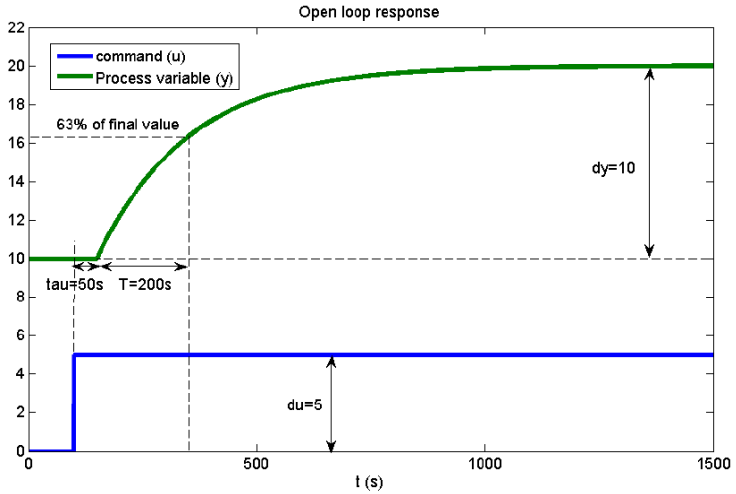
DRAFT

Open Loop Step Response

- A good starting point is to make an open loop step test on the process
- Note that this is a step on the controller output, not on the setpoint!

DRAFT

Open Loop Step Response



If our step response looks something like this, we have a stable system with a first order response

$$P(s) = \frac{K}{T \cdot s + 1} \cdot e^{-\tau \cdot s}$$

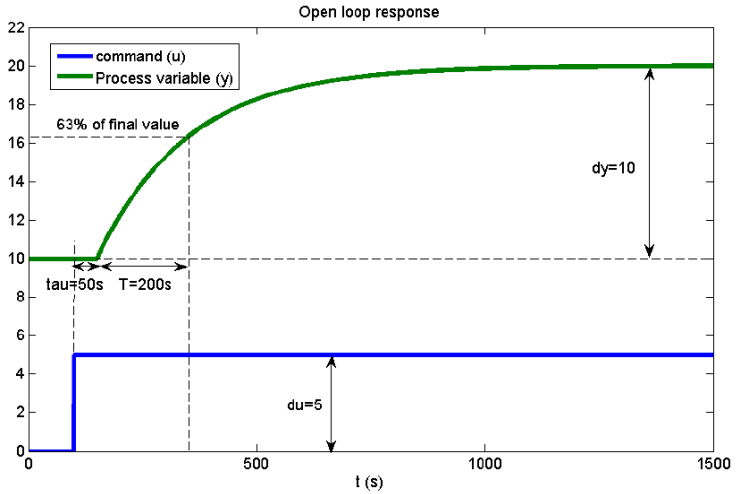
DRAFT

First Order System

$$P(s) = \frac{K}{T \cdot s + 1} \cdot e^{-\tau \cdot s}$$

We can get 3 pieces of information about the process:

- Process Gain K
- Time Constant T
- Time Delay τ



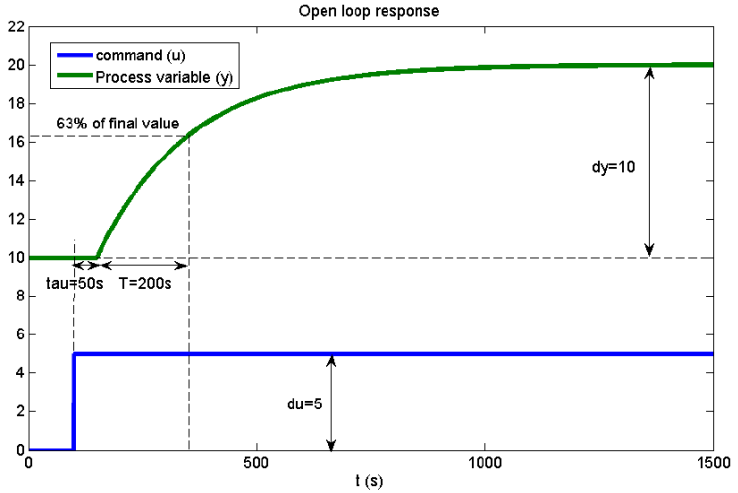
DRAFT

Process Gain

This is the ratio of the change in MV to the change in control input

$$K = \frac{\Delta Y}{\Delta U}$$

In this case K is (20-10)/(5-0) = **2**

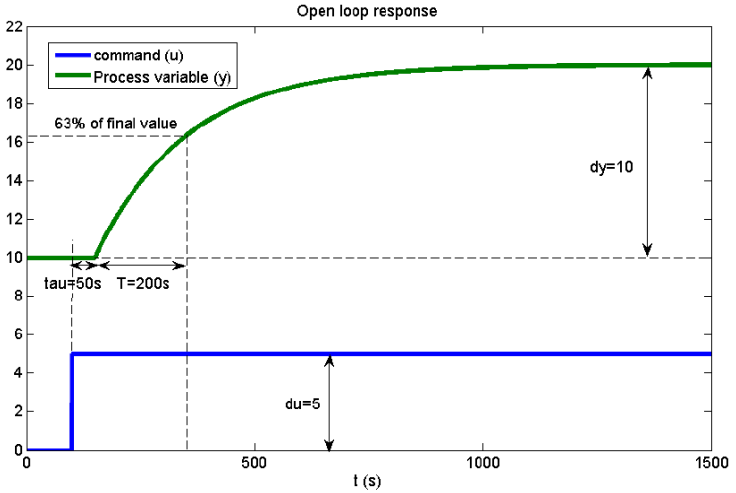


DRAFT

Time Constant

This is the ‘speed’ of the process.

To read it from the trend, look for the time it takes for the MV to rise to **63%** of its final value



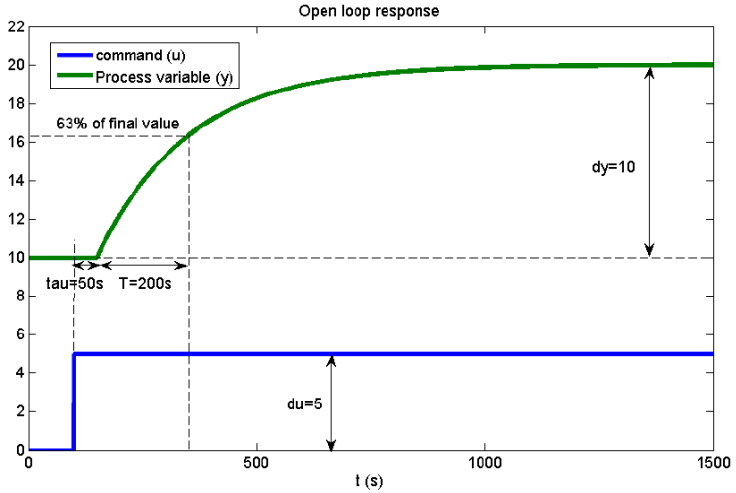
DRAFT

Time Constant

Ok, **why 63%??**

Because $1 - e^{-1}$ is about 0.63….

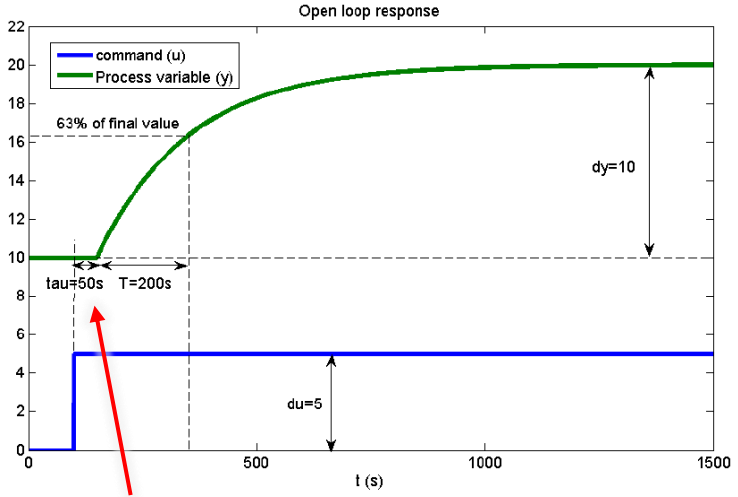
The step response is $1 - e^{-\frac{t}{T}}$ so after time T this is $1 - e^{-1}$



DRAFT

Time Delay

This is simpler, it is just the time it takes from the start of the step until the MV starts to move.



DRAFT

First order process model

We have 3 parameters that determine behavior

- K determines how big the output change will be
- T determines how long the process takes to get there
- τ determines how long it takes before the process starts doing anything at all

DRAFT

Implications for control

The key feature is the relation between T and τ

- If τ is small and T is big, control using PI is fine
- If τ is similar in size to T , we may need a more complex controller (PID)
- If τ is big relative to T , PID will struggle; it will integrate during the delay and then see a large jump!

DRAFT

Tuning for a first order process

We have a process that looks like

$$G(s) = \frac{K}{Ts + 1} e^{-\tau s}$$

And we know what K, T and τ are from our step test. Let's say that we want our controlled system to behave like this, but with a time constant $\boldsymbol{T_{cl}}$

DRAFT

Tuning for a first order process

Now we know what our closed loop should look like, and we also know what the process looks like, and what the controller looks like. We can use the closed loop relation:

$$G_{cl}(s) = \frac{C(s)G(s)}{1 + C(s)G(s)}$$

Where $C(s)$ is our controller.

DRAFT

Tuning for a first order process

$$G_{cl}(s) = \frac{C(s)G(s)}{1 + C(s)G(s)}$$

From here we can work out what controller parameters we need!

This is called **Internal Model Control (IMC)**

DRAFT

Internal Model Control (IMC)

Once we do the math, we get these parameters for PI:

$$K_p = \frac{1}{K}\frac{T}{(T_c + \tau)}$$

$$T_i = \min\left(T, 4(T_C + \tau)\right)$$

(this is Skogestad's IMC, S-IMC)

DRAFT

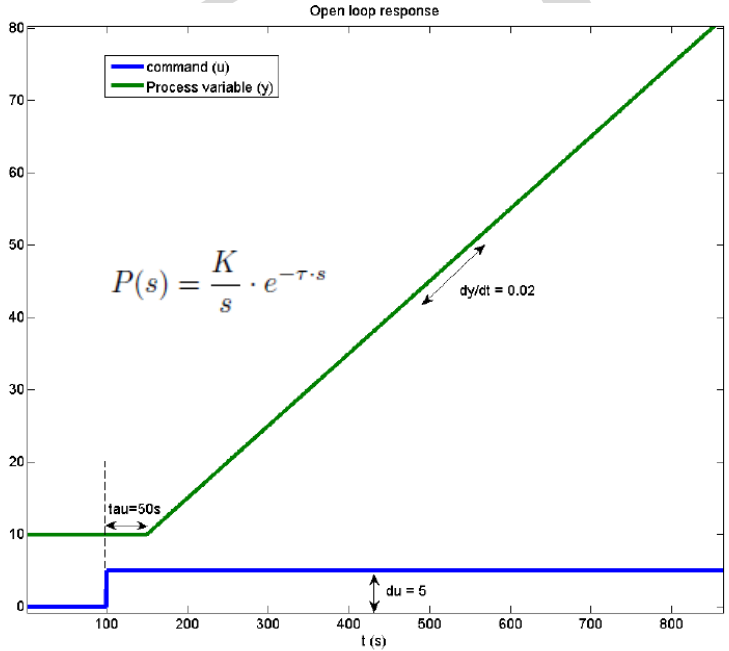
Internal Model Control (IMC)

IMC rules are nice, because they are not just heuristics: they are calculated from a desired behavior.

DRAFT

Integrating processes

- What happens when the step response is a ramp?
- Process contains an integrator!
- $K = \frac{dY/dt}{\Delta U}$



DRAFT

SIMC tuning rules

The rules can be derived for a number of different process models

Process	$g(s)$	K_c	τ_I	$\tau_D^{(5)}$
First-order, (5.3)	$k\frac{e^{-\theta s}}{(\tau_1 s+1)}$	$\frac{1}{k}\frac{\tau_1}{\tau_c+\theta}$	$\min\{\tau_1, 4(\tau_c+\theta)\}$	–
Second-order, (5.4)	$k\frac{e^{-\theta s}}{(\tau_1 s+1)(\tau_2 s+1)}$	$\frac{1}{k}\frac{\tau_1}{\tau_c+\theta}$	$\min\{\tau_1, 4(\tau_c+\theta)\}$	τ_2
Pure time delay[1]	$ke^{-\theta s}$	0	$0^{(*)}$	–
Integrating[2]	$k'\frac{e^{-\theta s}}{s}$	$\frac{1}{k'}\cdot\frac{1}{(\tau_c+\theta)}$	$4(\tau_c+\theta)$	–
Integrating with lag	$k'\frac{e^{-\theta s}}{s(\tau_2 s+1)}$	$\frac{1}{k'}\cdot\frac{1}{(\tau_c+\theta)}$	$4(\tau_c+\theta)$	τ_2
Double integrating[3]	$k''\frac{e^{-\theta s}}{s^2}$	$\frac{1}{k''}\cdot\frac{1}{4(\tau_c+\theta)^2}$	$4(\tau_c+\theta)$	$4(\tau_c+\theta)$
IPZ process[4]	$k'\frac{e^{-\theta s}}{s}\frac{Ts+1}{\tau_2 s+1}$	$\frac{1}{k'T}\cdot\frac{\tau_2}{\tau_c+\theta}$	$\min\{\tau_2, 4(\tau_c+\theta)\}$	–

(1) The pure time delay process is a special case of a first-order process with $\tau_1 = 0$

(2) The integrating process is a special case of a first-order process with $\tau_1 \to \infty$

(3) For the double integrating process, integral action has been added according to (5.22)

(4) For the integrating-pole-zero (IPZ) process, we assume that $T > \tau_2$. Then $(Ts+1)/s \approx T$ (rule T2) and the PI-settings follow

(5) The derivative time is for the series form PID controller in (5.1)

$(*)$ Pure integral controller $c(s) = \frac{K_I}{s}$ with $K_I = \frac{K_c}{\tau_I} = \frac{1}{k(\tau_c+\theta)}$

http://folk.ntnu.no/skoge/publications/2012/skogestad-improved-simc-pid/PIDbook-chapter5.pdf

DRAFT

SIMC for UNICOS PID

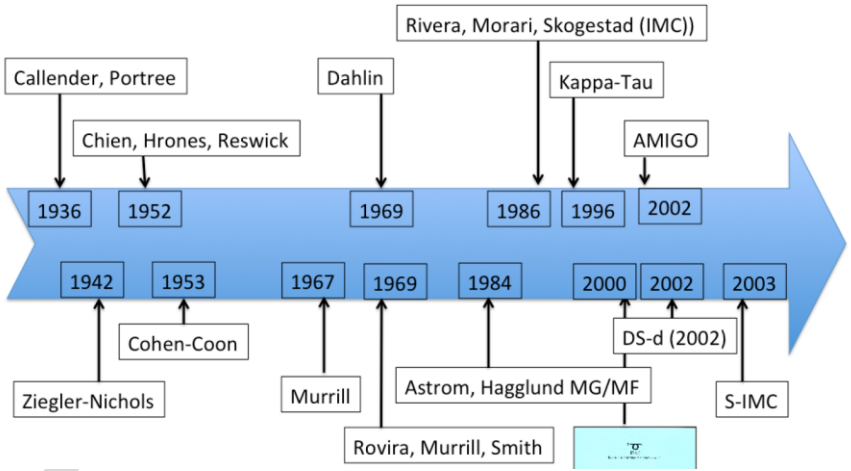
Point to note: SIMC rules apply for a 'cascade' PID structure, not the ISA standard used by UNICOS. For PI and PD they are the same, but for the full PID we need to do a transformation of the parameters:

$$K_p = K_p^c\left(1 + \frac{\tau_d^c}{\tau_i^c}\right), \tau_i = \tau_i^c\left(1 + \frac{\tau_d^c}{\tau_i^c}\right), \tau_d = \frac{\tau_d^c}{1+\frac{\tau_d^c}{\tau_i^c}}$$

DRAFT

Other tuning rules

- There are MANY other sets of tuning rules!



Ziegler-Nichols

- You may have come across Ziegler-Nichols
  - Early work (1942)
  - Actually developed for fast setpoint tracking, so they are really too aggressive for process control!
  - We will revisit them later!

DRAFT

Tuning rule warning

Lots of tuning rules are based on heuristics, which might only fit certain process types. In addition, you need to be sure what controller structure they are for. You also need to know what the control aim is (setpoint following, disturbance rejection)

DRAFT

Recommendation: SIMC

- SIMC is good because:
  - It forces you to do a step test and learn about the process
  - You can adjust the rules with an intuitive parameter
  - They are easily derived and therefore understood

DRAFT

Recommendation: SIMC

Recommendation is to start with SIMC.

If you're still not happy, you can use it as a baseline, and make manual adjustments from there.

Let's now make a tuning 'recipe'!

DRAFT

Tuning ‘recipe’

- Step 1: Preparation:
  - Make sure it’s ok to put the process into open loop
  - Make sure the process is settled (steady state)
  - Make sure you are close to your normal operating point

DRAFT

Tuning ‘recipe’

- Step 2: Open loop step test
  - Put the controller into Output Positioning
  - Make a small step in the output value
  - Observe the MV, make sure it doesn’t hit the limits!
  - Once the MV has settled, make a step back to where you started.
  - Put the controller back in regulation

DRAFT

Tuning ‘recipe’

- Step 3: Calculate process parameters
  - Find K, T and τ
    - $K = \frac{\Delta Y}{\Delta U}$ (watch out for scaling!)
    - T is the time taken to rise to 0.63 ΔY
    - τ is the time between your step input and the first change in MV

DRAFT

Tuning ‘recipe’

- Step 4: Calculate controller parameters
  - Use SIMC: choose closed loop time T_{cl}
  - Apply tuning rules:

$$K_p = \frac{1}{K}\frac{T}{(T_c + \tau)}$$

$$T_i = \min\left(T, 4(T_C + \tau)\right)$$

DRAFT

Tuning ‘recipe’

- Step 5: Set parameters and test
  - Apply the new parameters in the controller
  - Make a small set point change and see how the controller reacts
  - If too aggressive, increase T_{cl} and recalculate parameters
  - If too slow, decrease T_{cl} and recalculate parameters

DRAFT

Ziegler-Nichols method

1. Set K_d and K_i to **zero**.
2. Adjust K_p from 0 until the system starts to **oscillate** at certain frequency.
3. **Measure** the value $K_u = K_p$, and the oscillation period as T_u.
4. **Set** the various **gain factors** according to the follow formula and table:

$$u(t) = K_p(e(t) + \frac{1}{T_i}\int_0^t e(\tau)d\tau + T_d\dot{e}(t)$$

Control Type	K_p	T_i	T_d
P	$0.5K_u$	-	-
PI	$0.45K_u$	$T_u/1.2$	-
PD	$0.8K_u$	-	$T_u/8$
classic PID	$0.6K_u$	$T_u/2$	$T_u/8$

Manual method

1. Set K_p , K_d and K_i to zero.
2. Start with a small K_p, double it each time until the system starts to oscillate.
3. Half the value of K_p.
4. Start with a small K_d, double it each time until the system starts to oscillate.
5. Half the value of K_d .
6. Start with a small K_i, double it each time until the system starts to oscillate.
7. Half the value of K_i.

- If you are only using PD or PI controller, skip the irrelevant steps.
- Fine tune the various gain until you get the response you want.